



Ejercicio_1. (1 puntos)

Elije el orden asintótico, de menor a mayor, de las siguientes complejidades temporales:

- | | |
|-----------------------|---------------------|
| $f1(n) = 2^n$ | a) $f3, f2, f4, f1$ |
| $f2(n) = n^{3/2}$ | b) $f3, f2, f1, f4$ |
| $f3(n) = n * \log(n)$ | c) $f2, f3, f1, f4$ |
| $f4(n) = n^{\log(n)}$ | d) $f2, f3, f4, f1$ |

Razona tu respuesta y detalla el porqué de la misma.

Ejercicio_2. (2,5 puntos)

Se tiene un avión de transporte para ayuda humanitaria cuya capacidad de carga es de T toneladas y un conjunto de palets $c_1, \dots, c_n$, cuyos pesos respectivos (expresados también en toneladas) son $t_1, \dots, t_n$ y para los que se tiene una prioridad o beneficio al ser transportados de $b_1, \dots, b_n$, respectivamente y correspondiendo **la mayor prioridad al número mayor**. Teniendo en cuenta que la capacidad del avión es menor que la suma total de los pesos de los palets y que se pueden partir los palets de la siguiente forma (cargar el palet completo en el avión, no cargar o cargar solo la mitad del palet, $x_i=0$, $x_i=1/2$, $x_i=1$).

- (1,5 puntos). Diseñar un algoritmo con el esquema voraz, aunque no se garantice la solución óptima. Es necesario marcar en el código propuesto a qué corresponde cada parte en el esquema general de un algoritmo voraz (criterio, candidatos, función.....). Si hay más de un criterio posible elegir uno razonadamente y discutir los otros. Comprobar si el algoritmo garantiza la solución óptima en cada caso (la demostración se puede hacer con un contraejemplo).
- (0,5 puntos) Aplica el algoritmo para $n = 6$; $T = 100$; $t = (10, 5, 7, 20, 60, 35)$; $b = (20, 30, 20, 10, 500, 25)$
- (0,5 puntos) Analiza su complejidad.

Ejercicio_3. (2 puntos)

Se dispone de un algoritmo de reducción (AR) que resuelve un problema en tiempo dado por la siguiente ecuación de recurrencia $T(n) = 2T(n-1) + n$ con $T(0)=0$. Se ha desarrollado otra estrategia de Divide y Vencerás (DyV) para abordar el mismo problema. Esta estrategia realiza $n \log n$ operaciones para dividir el problema en dos subproblemas de tamaño $n/4$ y 4 de tamaño $n/8$ y $n \log n$ operaciones para combinar las soluciones de los subproblemas y obtener la solución del problema original.

- Calcular la eficiencia del primer algoritmo AR por el teorema maestro. (0,5 puntos)
- Calcular la eficiencia del algoritmo de DyV por algún método distinto del teorema maestro. (1,25 puntos)
- Estudiar cuál de los dos algoritmos es más eficiente. (0,25 puntos)

NOTA:

El Teorema maestro aplicado a $T(n) = aT(n-b) + \Theta(n^k)$ es:

$$T(n) \in \begin{cases} O\left(\frac{n}{a^b}\right) & \text{si } a > 1 \\ O(n^{k+1}) & \text{si } a = 1 \\ O(n^k) & \text{si } a < 1 \end{cases}$$



Ejercicio_4. (2 puntos)

Para el siguiente algoritmo, $A(n)$, definimos la función $t(n)$ como el número de líneas generadas por una ejecución del mismo:

```

procedure A(x: entero)
  para i=1 hasta x
    para j=1 hasta i
      GENERAR_LINEA(i, j, x)
    fpara
  fpara
  si x > 0 entonces
    para i= 1 hasta 4
      A(x div 2)
    fpara
  fsi
fprocedure
  
```

1. Calcula el orden de la función $t(n)$ (1,5 puntos)
2. Corroborar el resultado anterior aplicando el teorema maestro. (0,5 puntos)

NOTA:

Teorema: La solución a la ecuación $T(n) = aT(n/b) + \Theta(n^k \log^p n)$, con $a \geq 1$, $b > 1$ y $p \geq 0$, es:

$O(n^{\log_b a})$ si $a > b^k$

$O(n^k \log^{p+1} n)$ si $a = b^k$

$O(n^k \log^p n)$ si $a < b^k$

Ejercicio_5. (2,5 puntos)

Imagina que tienes una lista de datos de tamaño n que contiene números enteros. Deseas procesar esta lista utilizando un algoritmo de reducción multi-nivel que realiza múltiples reducciones en cada paso. El algoritmo funciona de la siguiente manera:

1. Procesa recursivamente la lista en tres partes de tamaño aproximado un tercio.
2. Luego procesa recursivamente cuatro listas de tamaño de aproximadamente un noveno de lista original
3. Finalmente, combina los resultados de todas las sublistas para obtener el resultado final.

1. Función de Recurrencia (0.5 puntos):

- Define la función de recurrencia para el algoritmo de reducción multi-nivel y explica detalladamente cada uno de los sumandos.

2. Complejidad Temporal (1 punto):

- Resuelve la recurrencia para determinar la complejidad temporal del algoritmo. Explica cada paso en tu razonamiento.

3. Implementación del Algoritmo (1 punto):

- Implementa el algoritmo de reducción multi-nivel en C++. Asegúrate de que el algoritmo siga las etapas descritas en el enunciado.